ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PAPER 2 · PRACTICE C

MARKS: 100 **TIME: 3 hours**

NC(V) Programming and Robotics, Level 4. Python practical in the style of the DHET second papers (100 marks): a Robot class with methods and a DeliveryRobot subclass, a temperature log saved to and loaded from a text file with error handling, and a menu-driven club-membership program with a dictionary, functions and input validation.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of THREE questions. Answer ALL the questions.
2. This is a practical paper. Use a Python 3 IDE (such as Thonny or IDLE). If you do not have a computer, write the programs neatly in your answer book.
3. Save each program with the question number and your name or examination number, and add your name or examination number as a comment on the first line.
4. Print each program with line numbers. Save your work regularly. Programs are not marked strictly on syntax.

QUESTION 1

> A company runs a fleet of small robots. Every robot has a name, a battery level (100% when full) and a total distance driven. Driving uses 2% of the battery for every kilometre. Delivery robots can also carry parcels.

1.1 Create a new file called Question1_YourName.py with your name or examination number as a comment on the first line. (1)

1.2 Create a class named Robot with a constructor that takes name and battery (default 100), and also sets an attribute distance to 0. (6)

1.3 Add a method drive(km). If the battery is too low for the trip (2% per km), display "Battery too low" and do not drive; otherwise reduce the battery and add km to distance. (8)

1.4 Add a method charge() that sets the battery back to 100. (2)

1.5 Add a method status() that returns a string such as "R1: battery 80%, 10 km driven". (4)

1.6 Create a class DeliveryRobot that inherits from Robot. Its constructor must call the parent constructor and add an empty list named parcels. Add a method load(parcel) that adds a parcel to the list. (6)

1.7 Test your classes: create a DeliveryRobot named "D1", load two parcels, drive 10 km, and display its status and its parcels. (3)

[30]

QUESTION 2

> A technician records the temperature of a robot's motor at different times. Each reading is saved in a text file called temps.txt as one line with the time and the temperature separated by a comma, for example: 08:00,24.5

2.1 Create a new file called Question2_YourName.py. (1)

2.2 Write a function save_reading(time, temp) that adds one reading as a new line at the end of temps.txt, without deleting the readings already in the file. (5)

2.3 Ask the user for THREE readings (time and temperature) and save each one with save_reading(). (5)

2.4 Write a function load_readings() that reads temps.txt and returns a list of (time, temperature) tuples, with the temperature as a float. If the file does not exist, the function must display "No readings yet" and return an empty list. (8)

2.5 Use load_readings() to display every reading, the average temperature and the highest temperature with the time at which it was recorded. (8)

2.6 Save and print the program, and a screenshot of the output. (3)

[30]

QUESTION 3

The college robotics club keeps a record of its members and the level each member has reached (1 to 4). The club secretary wants a menu-driven program to manage the members.

The following steps show how the program should work:

```
1 Add  2 Remove  3 Search  4 List  5 Quit
Choice: 1
Member name: Lerato
Level (1-4): seven
Please enter a number from 1 to 4.
Level (1-4): 3
Lerato added at level 3.
Choice: 4
Bongani - level 2
Lerato - level 3
Naledi - level 4
Thabo - level 1
Choice: 3
Name to search: Sipho
Sipho is not a member.
Choice: 9
Invalid choice.
Choice: 5
Goodbye!
```

3.1 Create a new file called Question3_YourName.py. (1)

3.2 Create a dictionary named members with three starting members: Thabo (level 1), Bongani (level 2) and Naledi (level 4). (3)

3.3 Write a function add_member() that asks for a name and a level, keeps asking until the level is a whole number from 1 to 4 (use try … except to handle text input), and adds the member. (8)

3.4 Write a function remove_member() that removes a member if the name is in the dictionary, or displays a message if it is not. (5)

3.5 Write a function search_member() that displays the level of a member, or a message if the name is not found. (4)

3.6 Write a function list_members() that displays all members in alphabetical order with their levels. (5)

3.7	Write the menu loop that calls the correct function for choices 1 to 4 and ends the program with "Goodbye!" for choice 5.	(8)
3.8	Display "Invalid choice." for any other input.	(2)
3.9	Test the program with the data in the example above and take a screenshot of the output.	(2)
3.10	Save and print the program with line numbers.	(2)
		[40]
		TOTAL: 100

END OF QUESTION PAPER